

Metodología de Sistemas I

Unidad N°1: “Herramientas de Repositorios de Software y Colaboración”

- 1. Definiciones y Naturaleza del Software.
- 2. Taxonomía del Software (Tipos de Sistemas).
- 3. Atributos de Calidad (Software de Excelencia).
- 4. Diferencias entre Ingeniería de Software y Ciencias de la Computación.

AGENDA

1. Definiciones y Naturaleza del Software

- ✓ Perspectiva de Pressman: Define el software como un elemento de sistema que es lógico, no físico. Subraya que el software no se "gasta" (como el hardware), sino que se deteriora debido a los cambios laterales que se introducen durante su mantenimiento.
- ✓ Perspectiva de Sommerville: Define a la Ingeniería de Software como una disciplina de la ingeniería que se interesa por todos los aspectos de la producción de software, desde las etapas iniciales de la especificación hasta el mantenimiento después de que el sistema entra en uso.

El Software no se "Fabrica", se Desarrolla

- ✓ Enfoque en el Diseño: El costo y la calidad del software se determinan casi exclusivamente en la etapa de diseño y construcción lógica.
- ✓ Personalización: A diferencia de una línea de montaje, la mayoría del software se construye a medida de una necesidad, incluso cuando se utilizan componentes preexistentes.

2. La Curva de Deterioro frente al Desgaste

- ✓ Hardware: El hardware falla mucho al principio (defectos de fábrica), luego entra en un periodo de estabilidad y finalmente se "gasta" por factores físicos (fricción, calor).
- ✓ Software: El software no se gasta. Sin embargo, su tasa de fallas aumenta cada vez que se le realiza un cambio. Esto sucede porque al corregir un error o agregar una función, se suelen introducir "efectos secundarios" no deseados en otras partes del sistema.

El Software como Entidad Evolutiva (Leyes de Lehman)

- ✓ Adaptabilidad: El software debe ser "maleable". Si la estructura interna del código (arquitectura) es rígida, el software muere.
- ✓ Complejidad Creciente: A medida que el software evoluciona, su complejidad aumenta a menos que se trabaje activamente en simplificarlo (refactorización).

Componentes de la Naturaleza del Software

- ✓ 1. Instrucciones (Algoritmos): El motor lógico que realiza las tareas.
- ✓ 2. Estructuras de Datos: La forma en que se organiza la información (vital en la era del Big Data y Cloud).
- ✓ 3. Documentación: Manuales, modelos UML y comentarios que permiten a los ingenieros entender la lógica meses o años después de haber sido escrita.

2. Taxonomía del Software (Tipos de Sistemas)

A. Según su Disponibilidad en el Mercado.

B. Según su Dominio de Aplicación.

C. Clasificaciones Modernas y Emergentes.

A. Según su Disponibilidad en el Mercado.

1. Productos Genéricos (Stand-alone): Son sistemas desarrollados por una organización para ser vendidos en un mercado abierto a cualquier cliente.
2. Productos Personalizados (A medida / Bespoke): Sistemas diseñados específicamente para un cliente particular que tiene una necesidad de negocio única.

B. Según su Dominio de Aplicación

1. Software de Sistemas: Programas diseñados para servir a otros programas. Interactúan directamente con el hardware y gestionan recursos compartidos.
2. Software de Ingeniería y Ciencias: Caracterizado por algoritmos de "manejo de números".
3. Software Incrustado (Embedded): Reside dentro de la memoria de solo lectura (ROM) de un sistema más grande. Controla funciones de un producto físico.
4. Software de Inteligencia Artificial: Utiliza algoritmos no numéricos para resolver problemas complejos que no admiten un análisis algorítmico directo.

C. Clasificaciones Modernas y Emergentes

1. **Sistemas Web y Aplicaciones Móviles:** Han pasado de ser simples hipertextos a aplicaciones transaccionales complejas que residen en la red pero ofrecen experiencias de usuario locales.
2. **Sistemas de Sistemas (SoS):** Son sistemas integrados por otros sistemas que son independientes entre sí, pero que colaboran para una meta mayor.
3. **3Computación Ubicua:** Software diseñado para redes de sensores inalámbricos y dispositivos portátiles (IoT), donde el software está en "todos lados" pero es invisible para el usuario.

3. Atributos de Calidad (Software de Excelencia)

A. Los "Cuatro Esenciales" de Sommerville.

B. La Visión de Pressman: Calidad Interna y Externa.

C. Factores de Calidad Adicionales (Estándar ISO/IEC 25010).

A. Los "Cuatro Esenciales" de Sommerville

1. Mantenibilidad (Maintainability): El software debe estar escrito de forma que pueda evolucionar para satisfacer las cambiantes necesidades de los clientes.

2. Confiabilidad y Seguridad (Dependability):

Confiabilidad: El sistema debe comportarse como se espera, sin fallos imprevistos.

Seguridad: Incluye la protección contra ataques (Security) y la garantía de que un fallo no causará daños físicos o económicos (Safety).

3. Eficiencia (Efficiency): El software no debe desperdiciar recursos del sistema, como memoria, ciclos de procesador o ancho de banda.

4. Aceptabilidad / Usabilidad (Acceptability): Debe ser comprensible para los usuarios finales, compatible con otros sistemas que ya utilicen y fácil de operar.

B. La Visión de Pressman: Calidad Interna y Externa

- ✓ Calidad de Diseño (Externa): Lo que el usuario percibe. Se refiere a los requisitos funcionales, la interfaz y el rendimiento.
- ✓ Calidad de Conformidad (Interna): El grado en que el código sigue los estándares de ingeniería (Clean Code, patrones de diseño). Un software puede parecer "bueno" por fuera pero estar "podrido" por dentro (deuda técnica).

B. La Visión de Pressman: Calidad Interna y Externa

- ✓ Calidad de Diseño (Externa): Lo que el usuario percibe. Se refiere a los requisitos funcionales, la interfaz y el rendimiento.
- ✓ Calidad de Conformidad (Interna): El grado en que el código sigue los estándares de ingeniería (Clean Code, patrones de diseño). Un software puede parecer "bueno" por fuera pero estar "podrido" por dentro (deuda técnica).

C. Factores de Calidad Adicionales (Estándar ISO/IEC 25010)

- ✓ Portabilidad: Facilidad con la que el software puede trasladarse de un entorno a otro.
- ✓ Reusabilidad: El grado en que partes del software pueden utilizarse en otros proyectos, reduciendo costos futuros.
- ✓ Interoperabilidad: La capacidad de dos o más sistemas para intercambiar información y utilizarla de manera efectiva .
- ✓ Escalabilidad: Capacidad del sistema para manejar un crecimiento significativo en el número de usuarios o datos sin degradar el servicio.

Cuadro de Prioridades según el Contexto

Tipo de Proyecto	Atributo Prioritario	Razón Técnica
App Bancaria	Seguridad / Confiabilidad	Protección de activos y datos sensibles.
Videojuego	Eficiencia / Usabilidad	Experiencia de usuario en tiempo real.
SaaS en la Nube	Escalabilidad / Mantenibilidad	Capacidad de crecer y actualizarse sin cortes.
Sistemas Médicos	Seguridad (Safety)	Un fallo pone en riesgo la vida del paciente.

4. Diferencias entre Ingeniería de Software y Ciencias de la Computación

- A. Enfoque Teórico frente a Enfoque Práctico.
- B. El Factor "Restricción" (Recursos y Tiempo).
- C. El Objeto de Estudio: Algoritmo vs. Sistema.
- D. Métodos de Trabajo.

A. Enfoque Teórico frente a Enfoque Práctico

- ✓ Ciencias de la Computación: Se centran en las teorías y fundamentos que subyacen a las computadoras y los sistemas de software.
- ✓ Ingeniería de Software: Se centra en las prácticas y problemas de la vida real para producir software útil.

B. El Factor "Restricción" (Recursos y Tiempo)

- ✓ En CC, un investigador puede buscar el "algoritmo perfecto" sin importar cuánto tiempo tome encontrarlo.
- ✓ En IS, el ingeniero debe encontrar la mejor solución posible dadas las restricciones de:
 - o Presupuesto limitado.
 - o Agenda (Deadlines).
 - o Recursos técnicos.

C. El Objeto de Estudio: Algoritmo vs. Sistema

- ✓ Ciencias de la Computación: A menudo trabajan con programas pequeños o piezas de lógica aisladas para demostrar un teorema o una eficiencia de cómputo.
- ✓ Ingeniería de Software: Trabaja con sistemas de gran escala. El desafío no es solo que el código funcione, sino cómo integrar millones de líneas de código escritas por diferentes personas, asegurar que sean mantenibles, seguras y que sobrevivan al paso del tiempo.

D. Métodos de Trabajo

Dimensión	Ciencias de la Computación	Ingeniería de Software
Meta principal	Descubrimiento de nuevo conocimiento.	Entrega de un producto funcional y de calidad.
Criterio de éxito	Elegancia matemática y prueba de concepto.	Satisfacción del cliente y cumplimiento de requisitos.
Herramientas	Matemáticas discretas, lógica, análisis algorítmico.	Metodologías (Scrum/Kanban), estándares (ISO), gestión de proyectos.
Evolución	Estable (las leyes lógicas no cambian rápido).	Altamente dinámica (herramientas y lenguajes cambian cada año).

